

Code review — academy

Runde de review pe proiectul tău, cea mai nouă sus.

Proiect: mini-ERP de academie, C# consolă, fără framework. Regula fișierului: runda nouă se adaugă **sus**.
Runde vechi rămân dedesubt, condensate.

Runda 3 — 2026-09-09 — sha 70360d6

Scop: verificat commit-ul 70360d6 „more user fixes” — reacția ta la runda 2, plus starea celor 8  rămase deschise. Metodă: `dotnet build` pe codul tău, apoi o **copie** a proiectului în afara repo-ului, reparată strict cât să pornească, ca să pot rula scenariile. Codul tău nu a fost modificat.

Poarta de compilare — **PICĂ**

```
Build FAILED.  
    44 Warning(s)  
     4 Error(s)
```

A cincea oară în șapte commit-uri.

```
ViewLogIn.cs(10,39) CS7036: There is no argument given that  
                    corresponds to the required parameter  
                    'repository' of UserService(UserRepository)  
ViewLogIn.cs(21,33) CS0122: 'UserService.repository' is  
                    inaccessible due to its protection level  
Teacher.cs(87,32)   CS1503: Argument 1: cannot convert from  
                    'System.Guid' to 'string'  
Teacher.cs(87,68)   CS1503: Argument 4: cannot convert from  
                    'string' to 'int'
```

Am reparat pe copie exact atât: constructorul din `ViewLogIn`, accesul la `repository`, și argumentul lipsă din `Teacher.FromText`. Trei linii. După ele buildul e verde — deci restul codului nou e sintactic corect, iar tot ce urmează în review e verificat **prin rulare**, nu prin citire.

Ce s-a schimbat, și e mult

Runda asta e prima în care partea de fond s-a mișcat serios:

- **B4 din runda 2 e închis pe bune.** `CreateUser` cu `if (request != null)` și lanțul de `as` a dispărut. În locul lui: `CreateStudent`, `CreateTeacher`, `CreateAdmin`, fiecare cu tipul lui, fiecare trecând prin `UserMapper`. Zero `as` pe request-uri. Exact fix-ul propus, aplicat fără să-l copiezi mecanic — l-ai extins și la `Update` și la toate cele trei tipuri.
- **M4 (jumătate) închis:** ordinea argumentelor la `Teacher` e acum corectă (`..., Salary, WorkHours, Password`).
- **M3 (jumătate) închis:** `ToTeacher` întoarce `Teacher`, nu `User`.
- **C1 (parțial):** cele ~120 de linii comentate din `Student`, `Teacher`, `Admin` și `UserService` sunt șterse.
- **UserMapper** are acum cele 12 metode complete, grupate pe tip, cu DTO-uri corecte per tip.

Serviciul de useri e, ca formă, cel mai curat cod din proiect. Problema e că nimic din el nu se execută vreodată, și mai jos e explicat de ce.

Ce a mers prost

Ai făcut un pas mare într-o direcție pe care n-am apucat s-o discutăm: `Student`, `Teacher` și `Admin` implementează acum `ITextMapper<Student>`, `ITextMapper<Teacher>`, `ITextMapper<Admin>` — adică **entitatea și-a devenit propriul mapper**. Intuiția de la bază e bună („fiecare tip știe cum arată linia lui în fișier”), și e chiar jumătatea corectă din T3. Dar felul în care ai legat-o face ca toate cele șase metode noi să fie cod pe care nimeni nu-l poate chema. Detaliile la B3.

Și în paralel: din cele 8 🚫 rămase deschise din runda 2, **niciunul** nu e închis.

🚫 Critice

B1 — Nu compilează. Iar cele două linii sunt din aceeași frază a review-ului trecut

`ViewLogIn.cs:10` · `:21` · `Users/Models/Teacher.cs:87`

CUM E ACUM: cele 4 erori de mai sus. Două dintre ele sunt în `ViewLogIn`, iar runda 2 se termina, la B1, cu fraza asta:

Aceeași regulă („am șters tipul, dar nu și utilizatorii lui”) lovește și în `ViewLogIn.cs:10` și `:21`, unde ai mutat metode din `UserService` în `UserRepository` fără să actualizezi apelantul.

Ai reparat linia `:21`. Linia `:10`, numită în aceeași propoziție, a rămas neatinsă.

DE CE: asta e tiparul pe care ți-l semnalez a treia oară, dar acum în forma lui cea mai curată: propoziția conținea **două** adrese, ai reparat-o pe prima pe care ai deschis-o. Nu e neatenție — e felul în care citești un review: ca pe o listă de locuri, nu ca pe o listă de reguli. Regula era „am șters/mutat ceva, deci trebuie să găsesc **toți** apelanții”. Aplicată o dată, s-a oprit.

Și reparația de la `:21` arată același lucru la nivel de design. Ai scris:

```
User user = service.repository.GetByFirstAndLastName(firstName, lastName);
```

`repository` e `private readonly` în `UserService` (`UserService.cs:12`) — de-aia `CS0122`. Ai încercat să treci **prin** serviciu ca să ajungi la repository, în loc să ceri serviciului ce-ți trebuie. `private` nu e o formalitate: el spune „cine e afară nu trebuie să știe că `UserService` ține datele într-un `UserRepository`”. În ziua în care le ții în SQL, `ViewLogIn` n-ar trebui să afle. Ecranul are voie să ceară *un user după nume*; n-are voie să știe **de unde** vine.

La `Teacher.cs:87` e altceva, și e greșeala cea mai ușor de prins: constructorul cere 8 valori (`id, firstName, lastName, email, age, salary, workHours, password`), tu îi dai 7. Ai uitat `workHours`. Compilatorul n-are cum să spună „îți lipsește al șaptelea argument” — el încearcă supraîncărcarea cu 7 parametri, cea fără `id`, și îți raportează că `Guid`-ul nu intră într-un `string`. **Eroarea pe care o vezi nu e greșeala pe care ai făcut-o;** e prima nepotrivire de tip din varianta pe care a ales-o el. Când vezi „cannot convert” pe primul argument al unui constructor, prima verificare e numărul de argumente, nu tipul lor.

FIX:

```
// ViewLogIn.cs:10
UserService service = new UserService(new UserRepository());

// ViewLogIn.cs:21 - metoda se pune in UserService, nu se ocoleste
User user = service.GetByFirstAndLastName(firstName, lastName);

// UserService.cs - metoda care lipseste
public User GetByFirstAndLastName(string firstName, string lastName)
{
    return repository.GetByFirstAndLastName(firstName, lastName);
}

// Teacher.cs:87 - al 7-lea camp
return new Teacher(Guid.Parse(cuv[0]), cuv[1], cuv[2], cuv[3],
    Int32.Parse(cuv[4]), Int32.Parse(cuv[5]),
    Int32.Parse(cuv[6]), cuv[7]);
```

B2 — (NEATINS din runda 2) Aplicația nu pornește. Iar dacă o faci să pornească, nimeni nu se poate loga — nici măcar studenții

Users/Mappers/UserTextMapper.cs:13-18 · Common/Repository.cs:57-65 · ViewLogIn.cs:27-29

CUM E ACUM: pe copia care compilează, prima tastă:

```
Unhandled exception. System.FormatException: Unrecognized Guid format.
  at UserTextMapper.FromText(String text) in UserTextMapper.cs:17
  at Repository`1.Read() in Repository.cs:63
  at Repository`1..ctor(ITextMapper`1, ...) in Repository.cs:13
  at UserRepository..ctor() in UserRepository.cs:11
  at ViewLogIn.Logger() in ViewLogIn.cs:11
```

Identice, linie cu linie, cu ce era în runda 2. Apoi am scos prefixele din `users.txt`, ca să treacă de `Guid.Parse` — și am încercat să mă loghez cu **un student**, `Scarlat / Stefan`:

```
=====LOG IN=====
Numele: Prenumele: System.ArgumentException: Error on user type
  at ViewLogIn.Logger() in ViewLogIn.cs:line 68
```

Runda trecută nu se putea loga adminul. Acum nu se poate loga **nimeni**. Aplicația nu mai are niciun drum funcțional de la pornire până la un ecran.

DE CE: `UserRepository` (:11) îi dă lui `Repository<User>` un `new UserTextMapper()`, iar `UserTextMapper.FromText` întoarce mereu `new User(...)`. Deci fiecare linie din `users.txt` devine un `User` gol de tip. În `ViewLogIn`, cele trei `as` întorc toate `null`, se cade pe `else`, `"Error on user type"`.

Ăsta e exact B3 din runda 2, nemișcat. Ce merită adăugat acum e altceva: **tu ai scris runda asta trei** `FromText` care ar fi rezolvat problema — `Student.FromText`, `Teacher.FromText`, `Admin.FromText`, fiecare construind tipul corect. Și totuși stack trace-ul arată `UserTextMapper.FromText`. Motivul e la B3, și e singurul lucru din review-ul ăsta pe care te rog să-l citești de două ori.

FIX: vezi B3. Fără el, nimic altceva din proiect nu se poate testa.

B3 — Cele șase metode noi din modele nu pot fi chemate de nimeni. `Save()` tot rade parolele și salariile

Users/Models/Student.cs:5 · :14 · :19 · Users/Models/Teacher.cs:6 · :77 · :83 ·
Users/Models/Admin.cs:5 · :55 · :61 · Common/Repository.cs:9

CUM E ACUM: am pornit copia reparată, am creat un profesor prin serviciul tău nou (deci prin `UserMapper.ToTeacher`, care funcționează corect), și am chemat `Save()` :

```
profesor creat: Radu salariu=5000 ore=30

--- users.txt dupa Save() ---
d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
...
9c99cd02-...,Alex,Grozavu,gicu@hotmail.com,48
2729eb34-...,Radu,Marin,radu@gmail.com,40
```

Profesorul e pe disc **fără salariu, fără ore, fără parolă**. Adminul și-a pierdut `10000` și `gicuEgrozav`. Aceeași pagubă ca runda trecută — deși acum există, scris de tine, un `Teacher.ToText` care produce toate cele 9 câmpuri, și un `Admin.ToText` cu toate cele 8.

Niciunul dintre ele nu e chemat vreodată. Am căutat: singurele apeluri de `ToText` / `FromText` din tot proiectul sunt `Repository.cs:48` și `Repository.cs:63`, iar acolo se cheamă `mapper.X(...)`, unde `mapper` e `UserTextMapper`.

DE CE: sunt trei lucruri suprapuse, și fiecare singur ar fi de ajuns.

1. `ITextMapper<Student>` nu e un `ITextMapper<User>`. `Repository<User>` cere în constructor un `ITextMapper<User>` (`Repository.cs:9`). `Student` implementează `ITextMapper<Student>`. Un `Student` este un `User`, dar de aici nu urmează că un „mapper de studenți” e un „mapper de useri” — și pe bună dreptate: un `ITextMapper<User>` promite că poate transforma **orice** `User` în text. `Student.ToText` nu poate transforma un `Admin`. Regula asta se numește **invarianță generică**: `Cutie<Pisică>` nu e `Cutie<Animal>`, pentru că în a doua ai voie să pui un câine. Compilatorul o aplică mereu, chiar când în cazul tău particular n-ar strica nimic.

2. `FromText` e metodă de instanță — ca s-o chemi, îți trebuie deja un obiect din tipul pe care vrei să-l fabrici. Ca să faci un `Student` din text, ai nevoie de un `Student` pe care să chemi `FromText`. De unde îl iei pe primul? Nu ai de unde. Metoda care naște un obiect nu poate trăi pe obiectul pe care îl naște. De-aia fabricile sunt `static` sau stau pe **alt** obiect decât cel produs.

3. `ToText(T item)` primește un parametru pe care nu-l folosește. Rulat:

```
Student s1 = new Student("Ana", "Pop", "ana@gmail.com", 20);
Student s2 = new Student("Ion", "Ionescu", "ion@gmail.com", 30);
Console.WriteLine(s1.ToText(s2));
// -> STUDENT,dc9ab21c-...,Ana,Pop,ana@gmail.com,20
```

I-am cerut textul lui `s2` și am primit textul lui `s1`. În corpul metodei scrie `Id`, `FirstName`, `Age` — adică `this.Id`, nu `item.Id`. Parametrul `item` e mort. Semnul ăsta — un parametru pe care implementarea îl

ignoră — e aproape întotdeauna dovada că **interfața nu se potrivește cu locul unde ai pus-o**:

`ITextMapper<T>` e scrisă pentru cineva care traduce *alțceva*, nu pentru cineva care se traduce pe sine.

Pune-le cap la cap și ai răspunsul la ce ai încercat de fapt să faci. Intuiția ta — „fiecare tip știe formatul lui” — e **corectă**, și e exact miezul lui T3. Doar că locul ei nu e în entitate, ci într-un `UserTextMapper` care se uită la primul câmp și decide pe cine cheamă:

```
public User FromText(string text)
{
    string[] cuv = text.Split(',');

    if (cuv[0] == "STUDENT") { return StudentFromText(cuv); }
    if (cuv[0] == "TEACHER") { return TeacherFromText(cuv); }
    if (cuv[0] == "ADMIN") { return AdminFromText(cuv); }

    throw new ArgumentException("Tip de user necunoscut: " + cuv[0]);
}
```

Asta e **Factory Method** — T3. Metoda întoarce `User` (deci `Repository<User>` e mulțumit), dar obiectul dinăuntru e un `Student` adevărat, iar `as` -ul din `ViewLogIn` îl vede. Un singur loc din tot programul are voie să știe ce înseamnă `"ADMIN"`.

FIX: scoate `ITextMapper<...>` din cele trei modele. Mută cele șase metode în `UserTextMapper`, ca metode private care primesc `string[] cuv` și întorc tipul lor, și lasă `FromText` / `ToText` publice să facă ramificarea de mai sus. Cele trei modele rămân doar cu date și reguli — cum sunt `Book` și `Course`.

B4 — Fiecare `FromText` nou contrazice `ToText` -ul scris cu patru rânduri mai sus

`Users/Models/Student.cs:14` vs `:19` · `Users/Models/Teacher.cs:77` vs `:83` ·

`Users/Models/Admin.cs:55` vs `:61`

CUM E ACUM: am luat un obiect, i-am cerut textul cu propriul `ToText`, și i-am dat textul înapoi propriului `FromText`. Aceași clasă, aceeași instanță:

```

=== Teacher ===
ToText    -> TEACHER,7489a262-...,Radu,Marin,radu@gmail.com,
           40,5000,30,parolabuna           (9 campuri)
FromText  -> FormatException: Unrecognized Guid format.

=== Admin ===
ToText    -> ADMIN,8c090b5d-...,Alex,Grozavu,
           gicu@hotmail.com,10000,48,gicuEgrozav
FromText  -> FormatException: Unrecognized Guid format.

```

Toate trei clasele. `ToText` pune prefixul de tip pe prima poziție; `FromText` face `Guid.Parse(cuv[0])`. Sunt la 4 rânduri distanță una de alta, în același fișier.

Și mai e o nepotrivire, în adâncime: `Teacher.ToText` scrie 9 câmpuri, `Teacher.FromText` citește 8 (a sărit `workHours` — vezi B1). `Admin.ToText` scrie 8, `FromText` citește 7.

DE CE: ăsta e B2 din runda 2, copiat de trei ori. Iar mecanismul l-am scris atunci și rămâne valabil: pentru compilator, un `string.Split` și o concatenare de string-uri sunt două operații fără nicio legătură între ele. Nimic din limbaj nu verifică vreodată că ce scrie una poate fi citit de cealaltă. Singurul lucru care le ține sincronizate ești tu.

Ce merită adăugat runda asta e cum se apără cineva de treaba asta, pentru că „fii mai atent” nu e o metodă. **Testul e round-trip-ul:** ia un obiect, scrie-l, citește-l înapoi, compară câmp cu câmp. Trei linii de cod care se scriu o dată și prind toate cele șase nepotriviri de mai sus deodată — inclusiv `workHours` -ul lipsă, pe care ochiul nu-l vede pentru că e o virgulă printre alte opt:

```

Teacher t  = new Teacher("Radu", "Marin", "radu@gmail.com",
                        40, 5000, 30, "parolabuna");
Teacher t2 = mapper.FromText(mapper.ToText(t));
Console.WriteLine(t.Salary == t2.Salary && t.WorkHours == t2.WorkHours
                  && t.Password == t2.Password);

```

Regula generală, și e una dintre cele care se plătesc toată viața: **când scrii o pereche de funcții care se anulează una pe alta — serializare/deserializare, encode/decode, criptare/decriptare — nu le scrii separat.** Le scrii pe amândouă, în aceeași ședință, și imediat după ele scrii proba că `f(g(x)) == x`. Cât timp proba lipsește, cele două jumătăți se depărtează una de alta la fiecare modificare, tăcut.

FIX: după ce muți totul în `UserTextMapper` (B3), scrie perechea `ToText` / `FromText` una sub alta, cu prefixul în amândouă, și numără câmpurile pe degete. Apoi rulează round-trip-ul de mai sus pentru toate trei tipurile, o dată, înainte de commit.

B5 — Admin : vârsta și salariul se inversează. E fix ce anunța M4 din runda 2

Users/Mappers/UserMapper.cs:67-68 · Users/Models/Admin.cs:12 · :19 · :65

CUM E ACUM: am cerut un admin de 48 de ani cu salariul 10000 — datele reale din `users.txt` :

```
var req = new AdminCreateRequest("Alex", "Grozavu", "gicu@hotmail.com",
                                48, 10000, "gicuEgrozav");
Admin a = UserMapper.ToAdmin(req);

Salariul trebuie sa fie cel puțin minim pe economie
cerut : Age=48      Salary=10000
stocat: Age=10000   Salary=48
```

Un admin de 10000 de ani, cu salariul 48. Iar `Admin.ToText` îl scrie pe disc în ordinea asta, deci după un `Save()` e definitiv.

Runda 2, M4, cuvânt cu cuvânt:

La Admin nu te va salva: Admin(Guid id, string firstName, string lastName, string email, int salary, int age, string password) — salary înaintea lui age, ambele int, singura clasă din proiect cu ordinea asta. Acolo o inversare se compilează perfect și produce un admin de 10000 de ani cu salariul 48. Pune age înaintea lui salary, ca la Teacher.

Ai reparat ordinea la `Teacher`. La `Admin` n-ai atins nimic — și acolo era avertismentul.

DE CE: merită insistat pe **de ce te-a salvat compilatorul la Teacher și nu la Admin**, pentru că e aceeași greșeală cu două destine.

La `Teacher`, ordinea greșită pune un `string` (parola) acolo unde se aștepta un `int` (orele). Tipurile nu se potriveau, deci `CS1503`, deci build roșu, deci ai reparat.

La `Admin`, `salary` și `age` sunt amândouă `int`. Pentru compilator, `ToAdmin(request.Age, request.Salary)` și `ToAdmin(request.Salary, request.Age)` sunt **exact același apel** — el nu știe că unul e o vârstă și celălalt un salariu. `int` e `int`. Verificarea de tipuri e o plasă cu ochiuri mari: prinde „string în loc de int”, nu prinde „int în loc de alt int”.

Singurul lucru care a încercat să-ți spună ceva a fost linia `Salariul trebuie sa fie cel puțin minim pe economie`, care s-a afișat pentru că $48 < 3500$. Adică validarea ta a detectat corect anomalia — și a lăsat-o să treacă, pentru că e un `Console.WriteLine` și nu un `throw`. Vezi B8: dacă acolo ar fi fost `throw`, bug-ul ăsta ar fi crăpat la primul apel, cu mesaj, în loc să se strecoare pe disc.

Iar remediul structural, cel care face greșeala **imposibilă**, nu doar reparată: pune-le în aceeași ordine în toată clasa. Un `Admin` are `age` înainte de `salary` peste tot — în ambii constructori, în `ToText`, în `FromText`, în

DTO-uri. Când toate locurile au aceeași ordine, n-o mai poți greși dintr-o scăpare, pentru că nu mai există o a doua ordine de ales.

FIX:

```
// Admin.cs – ambii constructori, age înaintea lui salary
public Admin(Guid id, string firstName, string lastName, string email,
             int age, int salary, string password)

public Admin(string firstName, string lastName, string email,
             int age, int salary, string password)
```

Apoi `UserMapper.ToAdmin` și `Admin.FromText` merg fără nicio schimbare, pentru că amândouă trimit deja `Age` înaintea lui `Salary`. Se repară un singur loc și cad toate trei.

B6 — Garda „se află deja în baza de date” nu se declanșează niciodată

`Users/Services/UserService.cs:30` · `:43` · `:56`

CUM E ACUM: am chemat `CreateStudent` de trei ori cu **exact aceleași** date:

```
apel 1: acceptat, Id=460f0e6c-36e5-4c06-84c8-c5ddf86b8928
apel 2: acceptat, Id=373811ad-d3f3-4a7d-9d54-3a7d00a027b4
apel 3: acceptat, Id=d47e5967-b611-406c-a126-835c47f91053
-> 3 studenti identici in repository, zero exceptii
```

Și după `Save()`, trei linii identice în `users.txt`, cu trei GUID-uri diferite.

DE CE: urmărește ordinea a două rânduri (`UserService.cs:28-30`):

```
Student student = UserMapper.ToStudent(request); // 1. construiește
if (repository.FindById(student.Id) != null)      // 2. caută ce a făcut
```

`ToStudent` cheamă `new Student(...)`, care ajunge la `User(string, string, string, int)`, care cheamă `this(Guid.NewGuid(), ...)`. Deci pe linia 1 s-a născut un GUID nou-nouț, pe care nu-l mai are nimeni pe lumea asta. Pe linia 2 îl cauți în repository. Nu-l găsești. Niciodată. Condiția e **întotdeauna falsă** — la fel de inutilă ca `if (request != null)` de la B4 din runda 2, doar că pe partea cealaltă.

Cauza mai adâncă: ai pus verificarea de unicitate pe **identitate**, când tu voiai unicitate pe **date**. `Id` e identitate: e felul în care programul spune „ăsta e același obiect ca ăla”, și e proaspăt prin definiție la creare. Datele sunt

altceva: nume, prenume, email. Întrebarea pe care voiai s-o pui e „mai am deja pe cineva cu emailul ăsta?”, și ea nu se pune niciodată pe `Id`.

Regula, ca s-o poți refolosi: **verifici duplicatul pe cheia pe care o dă utilizatorul, nu pe cea pe care ți-o dai singur.** `Id` -ul îl generezi tu, deci nu poate fi duplicat. Emailul îl scrie el, deci poate.

FIX: verifică înainte de a construi, pe un câmp din request:

```
public StudentCreateResponse CreateStudent(StudentCreateRequest request)
{
    if (repository.GetByEmail(request.Email) != null)
    {
        throw new ArgumentException("Exista deja un user cu acest email");
    }

    Student student = UserMapper.ToStudent(request);
    repository.Add(student);
    return UserMapper.StudentToCreateResponse(student);
}
```

`GetByEmail` se scrie în `UserRepository`, lângă `GetByFirstAndLastName`, la fel construită. Același tratament la `CreateTeacher` și `CreateAdmin` — sunt trei locuri cu aceeași greșeală.

B7 — (runda 1, a treia oară) Orice tastă care nu e cifră omoară aplicația

`ViewStudent.cs:35` · `:64` · `:88` · `:107`

CUM E ACUM: cele patru `Int32.Parse(Console.ReadLine())` sunt neatinse, în aceleași linii. N-am putut să le lovesc prin meniu runda asta, pentru că B2 nu mă lasă să ajung la ecranul studentului — dar codul e identic cu cel de la care am reprodus `FormatException` în rundele 1 și 2, deci constatarea rămâne.

DE CE: mecanismul e explicat în runda 1. Ce e nou e altceva. ăsta e al treilea review în care apare, iar fix-ul are două rânduri și e scris integral mai jos, de două ori. În aceeași perioadă ai scris 141 de linii de cod nou. Nu-ți cer să lucrezi mai mult; îți cer să lucrezi în altă ordine.

Regula pe care ți-am cerut-o la runda 2 și pe care o repet pentru că e singura care contează acum: **nu începi un task nou cât timp există un  deschis din runda anterioară.** Cele patru linii de aici sunt 10 minute.

FIX:

```

if (!Int32.TryParse(Console.ReadLine(), out tasta))
{
    InputGresit();
    continue;
}

```

B8 — (runda 2) **Salary** și **Password** avertizează și acceptă oricum

Users/Models/Teacher.cs:39 · :52 · Users/Models/Admin.cs:35 · :48

CUM E ACUM: reprodus, ambele clase:

```

Salariul trebuie sa fie cel putin minim pe economie
Parola trebuie sa aiba cel putin 8 caractere
[teacher] cerut salariu=1 parola=abc -> stocat salariu=1 parola='abc'
[admin]   cerut salariu=1 parola=abc -> stocat salariu=1 parola='abc'

```

Toate patru locurile, neatinsse.

DE CE: mecanismul e în runda 2 — `Console.WriteLine` nu întrerupe execuția, `throw` da. Ce s-a adăugat între timp e o dovadă că nu e o chestiune de stil: la B5, exact `Console.WriteLine` -ul ăsta a fost singurul lucru din tot programul care a observat că salariul unui admin e 48. Dacă ar fi fost `throw`, inversarea `age / salary` ar fi murit la primul apel, cu mesaj și stack trace. Așa, s-a afișat un rând printre altele și obiectul stricat a mers mai departe.

Validarea care nu oprește nu e validare — e un comentariu care se vede la rulare. Iar valoarea ei reală o vezi abia când prinde un bug pe care nu-l cauți.

FIX: `throw new ArgumentException(...)` în toate patru locurile. (Unificarea celor două perechi de reguli e T1b; până atunci, `throw`.)

🟡 Importante

- **M1 — (R2, NEATINS)** `UserRepository.FindById` ascunde metoda moștenită. `UserRepository.cs:13` · `warning CS0108`, tot acolo. `Repository<T>.FindById` (`Repository.cs:21`) face exact același lucru. Șterge-o.
- **M2 — (R2, NEATINS)** `Email` nu se curăță de spații. `User.cs:119`. Reprodus: `" ana@gmail.com "` → stocat identic, lungime 19. `string text = value.Trim();` (`:103`) e calculat și nefolosit; `email =`

`value;` ar trebui să fie `email = text;`. În plus, verificarea de lungime 7–40 (`:98`) se face pe `value` , deci înaintea lui `Trim` .

- **M3 — (R2, jumătate) `UserMapper` tot nu e `static` .** `UserMapper.cs:7` — clasa are 12 metode `static` și niciun câmp, dar e `public class` , deci se poate scrie `new UserMapper()` . `BookMapper.cs:6` e `public static class` .  `ToTeacher` întoarce acum `Teacher` .
- **M4 — `ToStudentUpdateResponse` primește `User` , frații lui primesc tipul concret.** `UserMapper.cs:28` — (`User user`) , în timp ce `ToTeacherUpdateResponse` (`:57`) și `ToAdminUpdateResponse` (`:86`) primesc `Teacher` / `Admin` . Trei metode-surori, una cu altă semnătură.
- **M5 — nimic nu mai cheamă `Save()` pentru useri.** `UserService` are `Create*` , `Update*` , `Delete` — toate lucrează doar în lista din memorie. La închiderea aplicației se pierde tot. (Chemarea automată e chiar **T2, Observer** — dar nu înainte de T3, altfel primul `Save()` strică `users.txt` , vezi B3.)
- **M6 — `DeleteUser` nu verifică dacă userul există.** `UserService.cs:109–113` — dacă `FindById` întoarce `null` , `repository.Remove(null)` nu face nimic și metoda raportează succes. Frații ei, `Update*` , verifică și aruncă. Aceeași clasă, trei metode care verifică și una care nu.
- **M7 — (R2 M5, NEATINS) `StudentCreateRequest` e în alt namespace decât frații lui.** `Users/Dtos/StudentCreateRequest.cs:1` → `...Users.Models.Students.Dtos` , restul de 15 în `...Users.Dtos` . De-aia `UserMapper.cs:3` , `UserService.cs:5` și acum și `UserRepository.cs:4` cară un `using` în plus. Namespace-ul urmează folderul.
- **M8 — (R1, NEATINS) `EnrolmentService.GetEnrolmentIdByStudentId` (`:26–40`) e cod mort.** Nechemată de nimeni.
- **M9 — (R1, NEATINS) `InscriereCurs` ocolește serviciul.** `ViewStudent.cs:239` — `enrolmentService.Enrolments.Add(...)` direct în lista internă.
- **M10 — (R1, NEATINS) modificarea unei cărți nu confirmă nimic.** `ViewStudent.cs:196` — `response` atribuit și nefolosit.
- **M11 — (R1, NEATINS) login-ul de admin/profesor iese tăcut.** `ViewLogIn.cs:39` / `:53` — tot `//viewAdmin` și `//viewTeacher` .
- **M12 — (R1, NEATINS) serviciile se instanțiază de mai multe ori.** `ViewStudent.cs:15` (`CourseService`) și `:17` (`EnrolmentService`) își citesc singure fișierul din constructor. `BookService`  primește repository-ul.

Cleanups

- **C1 — (parțial ) cod comentat.** Modelele și `UserService` sunt curățate  — dar au apărut 28 de linii noi comentate în `User.cs:136–163` . `git` ține tot; șterge-le.
- **C2 — (R1, a treia oară) încă nu există `.gitignore` .** `bin/` și `obj/` apar ca *untracked* la mine, acum, după build.
- **C3 — (R1, NEATINS) `Data/students.txt` e mort.** `grep students.txt` peste tot codul: 0 rezultate.
- **C4 — (R1, NEATINS) `Console.WriteLine(aux);` rămas din depanare,** `ViewStudent.cs:276` .
- **C5 — (R1, NEATINS) parole în clar în `Data/users.txt` (`gicuEgrozav`).**

- **C6 — 44 de warnings** (erau 43). Printre ele `CS8618` pe `Teacher.password`, care rămâne `null` dacă vreun constructor nu-l setează.
 - **C7 — using-uri noi nefolosite**, adăugate chiar în commit-ul ăsta: `UserService.cs:1` (`Books.Models`), `Teacher.cs:2` (`System.Collections.Generic`), `UserRepository.cs:4` (`Students.Dtos`). `Ctrl+K, Ctrl+E` în Visual Studio.
 - **C8 — cinci fișiere fără linie goală la final** (`\ No newline at end of file` în diff). Fiecare modificare viitoare a ultimei linii va apărea în `git diff` ca „am șters și am adăugat”, chiar dacă n-ai atins-o.
 - **C9 — numele metodelor din IMapper sunt inconsistente**: la creare `StudentToCreateResponse`, la modificare `ToStudentUpdateResponse`. Alege un tipar și ține-l în toate șase.
-

Starea celor din runda 2

#	Runda 2	Acum	Cum am verificat
B1	nu compilează (16 erori)	🔴 redeschis — 4 erori noi, 2 din ele în liniile numite explicit în R2	<code>dotnet build</code>
B2	aplicația nu pornește (<code>Guid.Parse</code>)	🔴 NEATINS	rulat: același <code>FormatException</code> , aceeași linie
B3	adminul devine student, parola dispare	🔴 NEATINS — acum nu se loghează nici studenții	rulat: <code>Scarlat/Stefan</code> → „Error on user type”
B4	<code>CreateUser</code> : condiția mereu adevărată	✅ ÎNCHIS — metoda e ruptă corect în trei	rulat: <code>CreateTeacher</code> produce un <code>Teacher</code>
B5	<code>Console.WriteLine</code> în loc de <code>throw</code>	🔴 NEATINS (B8)	rulat: salariu=1, parolă= <code>abc</code> , ambele acceptate
B6	<code>Int32.Parse</code> pe tastă	🔴 NEATINS (B7)	cod identic; meniul e inaccesibil din cauza B2
M1	<code>FindById</code> ascunde metoda moștenită	🔴 NEATINS	<code>warning CS0108</code>
M2	<code>Email</code> fără <code>Trim</code>	🔴 NEATINS	rulat: lungime 19
M3	<code>UserMapper</code> nu e <code>static</code> ; <code>ToTeacher</code> → <code>User</code>	⚠️ jumătate — <code>ToTeacher</code> ✅ , <code>static</code> ❌	
M4	ordinea argumentelor la <code>Teacher</code> / capcana la <code>Admin</code>	⚠️ <code>Teacher</code> ✅ / <code>Admin</code> 🔴 s-a împlinit (B5)	rulat: Age=48 → stocat 10000
M5	namespace <code>StudentCreateRequest</code>	🔴 NEATINS (M7)	
M6–M10	cod mort, <code>InscriereCurs</code> ,	🔴 NEATINS (M8–M12)	<code>grep</code>

#	Runda 2	Acum	Cum am verificat
	confirmare, login tăcut, servicii duble		
C1	cod comentat	⚠ modelele ✓ / User.cs ⚫ nou	
C2–C5	.gitignore , students.txt , aux , parole	⚫ NEATINS	
C6	warnings	⚠ 43 → 44	
C7	using -uri nefolosite	⚫ înrăutățit — 3 noi	

Din 6 ⚫ : 1 închis (B4), 5 deschise. Din 10 🟡 : 1 jumătate, 9 deschise. Din 7 🟢 : 1 parțial, 6 deschise.

Nu e o rundă slabă — B4 era cel mai greu dintre ele și l-ai închis pe fond. Dar e a treia rundă în care restanțele se adună mai repede decât se sting, și de aici încolo asta e problema numărul unu, nu codul.

Before / After pentru cele

#	Acum	Cum ar trebui
B1	<pre>new UserService(); service.repository.GetByFirstAndLastName(...)</pre>	<pre>new UserService(new UserRepository()); service.GetByFirstAndLastName(...), cu metoda adăugată în UserService</pre>
B1	<pre>new Teacher(g, c[1], c[2], c[3], int c[4], int c[5], c[6]) — 7 valori</pre>	<pre>new Teacher(g, c[1], c[2], c[3], int c[4], int c[5], int c[6], c[7]) — 8 valori</pre>
B2	<pre>UserTextMapper.FromText → new User(...)</pre>	ramificare pe <code>cuv[0]</code> → <code>Student</code> / <code>Teacher</code> / <code>Admin</code> (T3)
B3	<pre>class Student : User, ITextMapper<Student> public Student FromText(string text)</pre>	<pre>class Student : User metodele mutate în UserTextMapper, ca private static Student StudentFromText(string[] cuv)</pre>
B4	<pre>ToText : "TEACHER," + Id + ... (9 câmpuri) FromText : Guid.Parse(cuv[0]) (8 câmpuri)</pre>	ambele cu prefix, ambele cu 9 câmpuri, scrise una sub alta + o probă de round-trip înainte de commit
B5	<pre>Admin(Guid id, ..., int salary, int age, string password)</pre>	<pre>Admin(Guid id, ..., int age, int salary, string password) — aceeași ordine ca peste tot</pre>
B6	<pre>Student s = ToStudent(request); if (repository.FindById(s.Id) != null)</pre>	<pre>if (repository.GetByEmail(request.Email) != null) { throw ... } Student s = ToStudent(request);</pre>
B7	<pre>tasta = Int32.Parse(Console.ReadLine());</pre>	<pre>if (!Int32.TryParse(Console.ReadLine(), out tasta)) { InputGresit(); continue; }</pre>
B8	<pre>if (value < 4257) { Console.WriteLine("..."); } salary = value;</pre>	<pre>if (value < 4257) { throw new ArgumentException("..."); } salary = value;</pre>

Întrebări de verificare

1. **B3.** `Student` este un `User`. De ce, atunci, `ITextMapper<Student>` **nu** e un `ITextMapper<User>`? Răspunde cu un exemplu concret: ce s-ar putea cere unui `ITextMapper<User>` și un `Student` n-ar putea face?
2. **B3.** `FromText` e metodă de instanță pe `Student` și trebuie să producă un `Student`. Scrie linia de cod care ar chema-o prima dată, la pornirea aplicației, când în memorie nu există încă niciun `Student`. Dacă nu poți — spune în ce constă imposibilitatea.
3. **B5.** Aceeași greșală — argumente în ordine inversă — te-a oprit la `Teacher` și a trecut la `Admin`. Explică diferența. Apoi: mai există în proiect vreo altă pereche de parametri vecini cu același tip, unde s-ar putea întâmpla la fel? Caută-i, nu ghici.
4. **B6.** `if (repository.FindById(student.Id) != null)` e mereu falsă; `if (request != null)` de runda trecută era mereu adevărată. Amândouă arată ca niște verificări normale. Ce au în comun, ca greșală de gândire? Și ce întrebare ți-ai putea pune, în viitor, ca să le prinzi la scris?
5. **Proces.** Runda 2 ți-a lăsat 8 🚫; ai închis 1 și ai deschis 5 noi. Fix-urile pentru B7 și B8 sunt scrise integral, cu cod, în două review-uri consecutive și n-ar lua împreună 20 de minute. Nu te întreb de ce n-ai apucat. Te întreb altceva: **ce ai face diferit ca să nu mai depindă de „am apucat”?** Răspunsul trebuie să fie un obicei, nu o intenție.

Ce urmează

Runda asta ordinea contează mai mult decât conținutul. Nimic din listă nu se poate verifica până nu merg primele două.

1. **Fă-I să compileze** — B1. Trei linii.
2. **Închide restanțele mecanice, în aceeași ședință:** B7 (4 linii), B8 (4 linii), M1, M2, M6, C2, C3, C4, C7. Toate au fix-ul scris. Împreună: sub o oră. Commit.
3. **Fă-I să pornească — T3, Factory Method.** B2 + B3 + B4 sunt același task: `UserTextMapper.FromText` se uită la `cuv[0]` și fabrică tipul potrivit; cele șase metode din modele se mută acolo; `ITextMapper` iese din `Student` / `Teacher` / `Admin`. DoD: pui un TEACHER în mijlocul lui `users.txt`, pornești, te loghezi cu el, salvezi, repornești — profesorul e tot profesor și are tot parola. Testul ăsta pică de trei runde.
4. **B5 și B6** — după ce aplicația pornește, ca să le poți verifica prin meniu.
5. Abia apoi **Faza 1, pașii 1 și 2** — `Course` și `Enrolment` (T0.5 → T0.6 → T1), care sunt încă neatinși din runda 2.

Un commit per pas. **Build verde înainte de fiecare commit** — e a cincea oară când o scriu, și e singura regulă din review care nu costă nimic.

Runda 2 — 2026-09-09 — sha 3c1877d

Verificat pasul 3 din Faza 1 (`User` → T0.5). Buildul a picat cu 16 erori (6 vizibile, 10 ascunse în spatele lor). Pașii 1 și 2 — `Course` și `Enrolment` — sări.

#	Constatare	Stare azi
B1	nu compilează: metode duplicate în <code>UserMapper</code> + DTO-uri șterse fără apelanți	🔴 redeschis (R3 B1)
B2	<code>UserTextMapper.FromText</code> face <code>Guid.Parse</code> pe prefixul de tip → aplicația nu pornește	🔴 NEATINS (R3 B2)
B3	<code>FromText</code> întoarce mereu <code>User</code> → adminul devine student, <code>Save()</code> rade parola și salariul	🔴 NEATINS (R3 B2, B3)
B4	<code>CreateUser</code> : <code>if (request != null)</code> mereu adevărat → orice cerere produce un <code>Student</code>	✅ ÎNCHIS
B5	<code>Salary</code> / <code>Password</code> în <code>Teacher</code> și <code>Admin</code> : <code>Console.WriteLine</code> în loc de <code>throw</code>	🔴 NEATINS (R3 B8)
B6	<code>Int32.Parse</code> pe tastă în <code>ViewStudent</code>	🔴 NEATINS (R3 B7)
M1	<code>UserRepository.FindById</code> ascunde metoda moștenită (<code>CS0108</code>)	🔴 NEATINS
M2	<code>Email</code> nu se face <code>Trim</code> , deși <code>FirstName</code> / <code>LastName</code> da	🔴 NEATINS
M3	<code>UserMapper</code> nu e <code>static</code> ; <code>ToTeacher</code> întoarce <code>User</code>	⚠️ jumătate
M4	ordinea argumentelor: <code>Teacher</code> greșit, <code>Admin</code> — capcană anunțată	⚠️ <code>Teacher</code> ✅ / <code>Admin</code> 🔴 (R3 B5)
M5	<code>StudentCreateRequest</code> în alt namespace decât frații lui	🔴 NEATINS
M6–M10	cod mort, <code>InscriereCurs</code> ocolește serviciul, modificarea	🔴 NEATINS

#	Constatare	Stare azi
	cărții nu confirmă, login admin/profesor tăcut, servicii instanțiate de mai multe ori	
C1–C7	cod comentat, <code>.gitignore</code> , <code>students.txt</code> mort, <code>Console.WriteLine(aux)</code> , parole în clar, warnings, <code>using</code> - uri	⚠ C1 parțial; restul NEATINSE

Runda 1 — 2026-09-08 — sha `79abec3`

Verificat T0.0–T0.4 din `TASKS.md`. Buildul a trecut. **5/5 task-uri închise pe fond.**

#	Constatare	Stare azi
T0.0–T0.4	nu compilează · „Vezi cursurile” crapă · modificarea orfanizează cartea · <code>createdAt</code> ignorat · <code>CreateUser</code> face mereu un <code>User</code> gol	✅ toate 5 închise
B1	<code>Contains</code> la căutarea după nume → Enter pe gol înseamnă „primul din listă”	✅ ÎNCHIS
B2	<code>null</code> strecurat în lista de cursuri	✅ ÎNCHIS
B3	profesorul salvat nu mai poate fi citit (<code>ToText</code> ≠ constructorul din text)	❌ reapărut de 3 ori în modele (R3 B4)
B4	<code>Int32.Parse</code> pe tastă	❌ NEATINS de 2 runde (R3 B7)
M1	<code>Create</code> / <code>Update</code> din modele = cod mort	✅ șterse
M2	<code>BookUpdateRequest</code> cere o dată ignorată	✅ ÎNCHIS de T0.5
M3	<code>GetEnrolmentIdByStudentId</code> cod mort	❌ NEATINS
M4	modificarea cărții nu confirmă nimic	❌ NEATINS
M5	<code>User newUser = new();</code> inutil	✅ ÎNCHIS
M6	<code>InscriereCurs</code> ocolește serviciul	❌ NEATINS
M7	login admin/profesor iese tăcut	❌ NEATINS
M8	serviciile se instanțiază de mai multe ori	⚠️ <code>BookService</code> ✅, restul ❌
C1–C7	<code>Console.WriteLine(aux)</code> · <code>.gitignore</code> · <code>students.txt</code> ·	✅ C6; restul ❌

#	Constatare	Stare azi
	parole în clar · warnings · override gol · using -uri	



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro